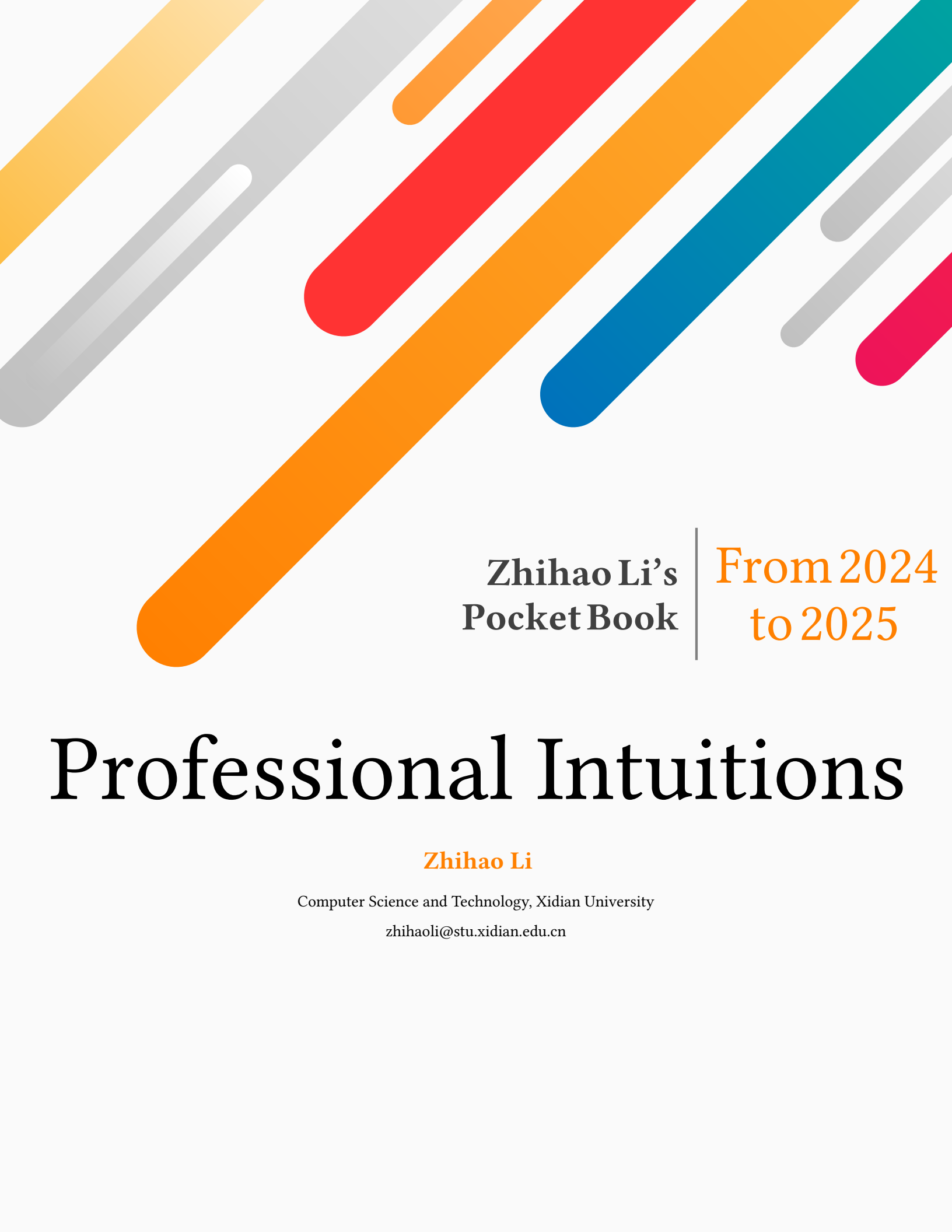




Zhihao Li's
Pocket Book

From 2024
to 2025

Professional Intuitions

Zhihao Li

Computer Science and Technology, Xidian University

zhihaoli@stu.xidian.edu.cn

CONTENTS

CHAPTER 1

PROFESSION & LEARNING

PAGE 5

1.1	Mathematical Principles	5
1.1.1	仿射变换 (Affine Transformation)	5
1.1.2	非线性频率压缩	5
1.1.3	离散 Bucket 设计	5
1.1.4	哈达玛积 (Hadamard product)	6
1.1.5	高斯分布	6
	高斯分布信噪比	6
1.2	Computer Graphics	6
1.2.1	Camera Theory	6
	相机成像	6
	相机内参	7
	视场角 (FoV)	7
	球谐函数 (Spherical Harmonics)	8
	关键帧	9
	UV 坐标系统	9
1.3	Machine Learning	10
1.3.1	Contrastive Learning	10
1.4	Digital Human	10
1.4.1	Model Learning	10
	FLAME	10

CHAPTER 2

PHILOSOPHY & INSIGHT

PAGE 11

2.1	Research Guidelines	11
2.1.1	Research Purpose	11
2.1.2	Global Stages	11
2.1.3	Early Stage	11
2.1.4	Research Survey	12
	Purpose	12
	Roadmap	12
	Survey Stage	12
2.2	Research Habits	13
2.2.1	Paper Reading	13
	跟进前沿	13
	论文略读	13
	论文精读	13
	论文总结	13
2.2.2	Paper Writing	14
	Abstract	14

2.2.3	Experiment Exploration	14
2.2.4	Balanced Life	14
2.2.5	Future Planning	14

CHAPTER 3 VIEWS & INSPIRATION PAGE 15

3.1	Audio-driven Methods with 3D Geometry	15
3.1.1	Mesh/Template-based	15
	<i>[TOG 2017 ADFA] [Regressive Models] [Lip Synchronization] [Expression]</i>	15
	<i>[CVPR 2019 VOCA] [Regressive Models] [Lip Synchronization] [Expression]</i>	16
	<i>[CVPR 2023 CodeTalker] [Generative Models] [Lip Synchronization] [Expression]</i>	16
	<i>[ICCV 2025 GaussianSpeech] [Generative Models] [Lip Synchronization] [Expression]</i>	17
	<i>[TJCAI 2025 GLDiTalker] [Generative Models] [Lip Synchronization] [Expression]</i>	17
3.1.2	Keypoints	19
3.1.3	3DMMs	19
3.1.4	Hybrid Prior	19
3.1.5	Nerf-based	19
3.1.6	3DGS-based	19
3.2	Benchmarking	19
3.2.1	Datasets	19
3.2.2	Evaluation Metrics	19
	<i>PSNR</i>	19
	<i>SSIM</i>	19
	<i>LPIPS</i>	20

CHAPTER 4 CODING & SKILLS PAGE 21

4.1	Environment Configuration	21
4.1.1	<i>Miniconda</i> Installation On Linux	21
4.1.2	<i>Pytorch3d</i> Installation on Windows	21
4.1.3	<i>Ninja</i> Installation on Linux	22
4.2	Python Debugging	22
4.2.1	<i>Visual Studio Code</i> Debugging Configuration	22
4.3	Bugs with Solution	24
4.3.1	Version Conflict	24
4.4	Coding Tricks	25
4.4.1	Python	25
	<i>pathlib</i>	25
	<i>8_000</i>	25
	<i>parser</i>	25
	<i>with</i>	26
	<i>tensor.scatter_add_</i>	26
	<i>register_buffer</i>	26
	<i>@dataclass</i>	27
	<i>field</i>	29
	<i>tyro.cli</i>	30
	<i>setattr</i>	30
	<i>torch.autograd.set_detect_anomaly</i>	31

	<i>pprint</i>	31
	<i>pathlib</i>	31
4.4.2	Cmd On Windows	32
	<i>pwd</i>	32
	<i>taskkill</i>	32
4.4.3	Linux Usage	32
	<i>whereis</i>	32
	<i>which</i>	32

CHAPTER 5

GRAMMAR & WRITINGS

PAGE 33

5.1	Mathematical Symbols in LaTeX	33
5.1.1	集合及向量空间的表示	33
5.1.2	运算符号	33
5.1.3	多行公式	33
5.2	Extended Markdown	34
5.3	Elaborating Ideas	35

CHAPTER 6

TEMPLATES & TOOLS

PAGE 36

6.1	Templates	36
6.1.1	Loss Calculation	36
	<i>L1 Loss</i>	36
	<i>L2 范数</i>	36
	<i>L3 范数</i>	37
	<i>Scaling Loss</i>	37
	<i>Cosine Similarity Loss</i>	37
	<i>Pixel-level Reconstruction Loss</i>	38
	<i>Perceptual Loss</i>	38
6.1.2	Computer Graphics	39
6.1.3	Setup Random Seed	40
6.1.4	Load and Check Model	40
6.2	Tools	41
6.2.1	FFmpeg	41
	编码器选择	42
	质量因子调节	42
	预设模式优化	42
	分辨率缩放策略	42
	关键帧控制	42
	元数据优化	43
6.2.2	SwanLab	43

Chapter 1

Profession & Learning

1.1 Mathematical Principles

1.1.1 仿射变换 (Affine Transformation)

$$price = w_{area} \cdot area + w_{age} \cdot age + b \quad (1.1)$$

仿射变换的特点是通过加权和对特征进行线性变换，并通过偏置项进行平移。

1.1.2 非线性频率压缩

在滤波器设计中将整个模拟频率轴压缩到 π/T 之间，使得 $H_a(s), s = j\Omega$ 压缩为 $\widehat{H}_a(s_1), s_1 = j\Omega_1$ ，可以利用正切变换实现频率压缩模型：

$$\Omega = \frac{2}{T} \tan\left(\frac{1}{2}\Omega_1 T\right) \quad (1.2)$$

这个设计思想实质上利用了正切函数定义域有限、值域无限以及奇函数的性质；推而广之，这种设计可以实现特定的单值压缩方法，也可以实现值域的延展。

1.1.3 离散 Bucket 设计

一些类似的函数特性，对数函数，指数函数分别适合于定义域、值域取值 $0 \sim 1$ 之间的情况，但是对目标域都有所限制，因此这些函数往往没有正切函数具有优良的特性。

对于一个连续角速度变量 w ，将其序列范围划分为 d 个大小不同的离散速度 buckets。每一个 bucket 都有一个中心 $c_i \in \{c_1, \dots, c_d\}$ 以及一个半径 $r_i \in \{r_1, \dots, r_d\}$ ，为了表示不同 bucket 的重要性，可以将角速度变量 w 转换为向量 $\vec{s} \in \mathcal{R}^d$ 表示，其中第 i 个值记为，

$$s_i = \tanh\left(\frac{w - c_i}{r_i} * 3\right) \quad (1.3)$$

其中， $\tanh(x)$ 函数是双曲正切函数 (hyperbolic tangent function)，值域是 $(-1, 1)$ 且单调递增，常用作激活函数，其定义为，

$$\tanh(x) = \frac{\sinh(x)}{\cosh(x)} = \frac{e^x - e^{-x}}{e^x + e^{-x}} \quad (1.4)$$

这种 bucket 方式用于将一个连续变量转换为特征向量表示，并且对于 $(w - c_i)/r_i$ 考虑了 w 距离 bucket c_i 的相对距离，而非绝对距离以保证对不同 bucket 具有一致的衡量标准。

1.1.4 哈达玛积 (Hadamard product)

Hadamard 乘积是矩阵的一类运算，对形状相同的矩阵进行运算，并产生相同维度的第三个矩阵。在数学中，Hadamard 乘积（也称为 Schur 乘积或逐元素乘积）是一种二元运算，它用两个具有相同维数的矩阵产生另一个具有相同维数的矩阵，其中每个元素 (i, j) 是原始两个矩阵的元素 (i, j) 的乘积。它是由法国数学家雅克·哈达玛 (Jacques Hadamard) 或德国数学家 Issai Schur 命名的。

$$C = (A \odot B)_{ij} = a_{ij} * b_{ij} \quad (1.5)$$

1.1.5 高斯分布

高斯分布信噪比

对于服从高斯分布 $\tau \sim N(\alpha, \sigma^2)$ 的随机变量而言，其均值被视为信号，而将方差视为噪声。因此，其信噪比 (SNR, Signal to Noise Rate) 计算为：

$$SNR = \frac{P_{signal}}{P_{noise}} = \frac{\alpha^2}{\sigma^2} \quad (1.6)$$

1.2 Computer Graphics

1.2.1 Camera Theory

相机成像

相机成像过程涉及到四个坐标系的变换，世界坐标系经过刚体变换（缩放、旋转、平移）后变成了相机坐标系，再次经过透视投影变成图像坐标系，最后经过仿射变换变成像素坐标系。

$$Z \begin{pmatrix} u \\ v \\ 1 \end{pmatrix} = \underbrace{\begin{pmatrix} \frac{1}{dX} & -\frac{\cot\theta}{dX} & u_0 \\ 0 & \frac{1}{dY\sin\theta} & v_0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} f & 0 & 0 & 0 \\ 0 & f & 0 & 0 \\ 0 & 0 & 1 & 0 \end{pmatrix}}_{\text{内参矩阵}} \underbrace{\begin{pmatrix} R & T \\ 0 & 1 \end{pmatrix}}_{\text{外参矩阵}} \begin{pmatrix} U \\ V \\ W \\ 1 \end{pmatrix}$$

仿射变换 透视投影 刚体变换

内参矩阵 外参矩阵

CSDN @思妙想多多财-杰

Figure 1.1: 坐标系变换过程

为了简洁表示，一般都通过内参矩阵和外参矩阵来实现坐标系的转换，内参矩阵就是相机的内参，外参矩阵就是相机的外参。

相机内参

相机内参包括相机的焦距（Focal Length）以及主点（Principal Point），焦距指的是成像元件的光学中心到成像平面的距离，单位通常是像素；主点是相机图像平面上的主点坐标，也就是图像坐标系的原点，通常是相机的光学中心在图像平面上的投影。主点是相机图像中理想情况下图像中心的位置，通常接近图像的几何中心，但在实际相机中，由于制造偏差，可能不完全重合。

针孔相机模型

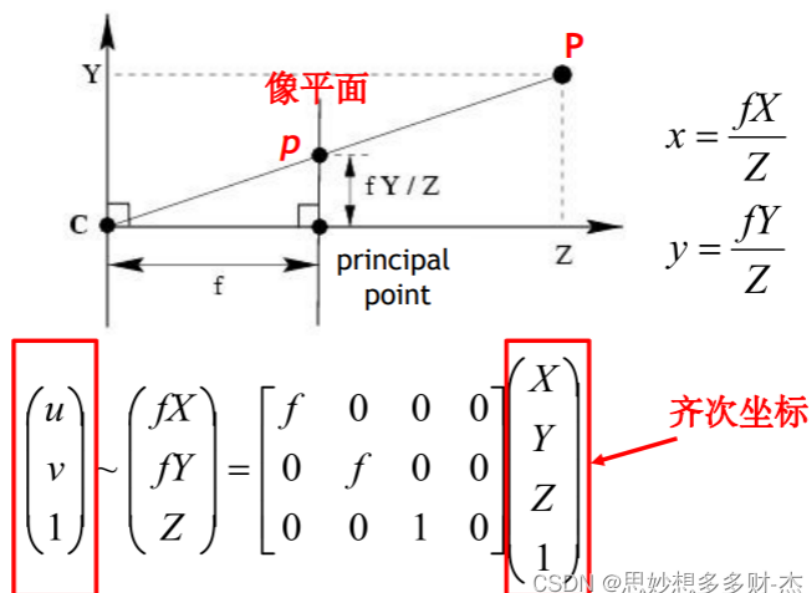


Figure 1.2: 相机内参计算

根据图 1.2 可以计算出对应的内参矩阵，一般来说， f_x 和 f_y 在一个理想的相机中应该是相同的，但是由于像素的非方形（长宽比不一致）或者相机的几何畸变，这两个值通常是不同的。它们反映了每个像素在水平方向和垂直方向上对应的实际焦距，如公式 1.7 所示。

$$K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \quad (1.7)$$

- f_x 和 f_y 分别是相机的横向和纵向焦距，在像素单位下的水平和垂直方向上的投影比例系数，表示了相机成像的缩放效果，通常以像素单位进行测量；当像素单元是正方形时， $f_x = f_y$ ，是长方形时， $f_x \neq f_y$ 。
- c_x 和 c_y 是相机光心在像素坐标系下的水平和垂直方向上的位置，表示了相机成像中心在图像上的位置。

视场角 (FoV)

视场角是以光学仪器的镜头为顶点，以被测目标的物像可通过镜头看到的最大范围的两条边缘构成的夹角，称为视场角（Field of View, FoV）。FoV 指的是相机视场张角，用来描述拍摄的范围。

最常用的是水平视场角（VFOV）和垂直视场角（HFOV），其与焦距的转换关系如下公式所示。

$$\begin{cases} \tan \frac{VFOV}{2} = \frac{w/2}{f_x} \Rightarrow f_x = \frac{w}{2 \tan \frac{VFOV}{2}} \\ \tan \frac{HFOV}{2} = \frac{h/2}{f_y} \Rightarrow f_y = \frac{h}{2 \tan \frac{HFOV}{2}} \end{cases} \quad (1.8)$$

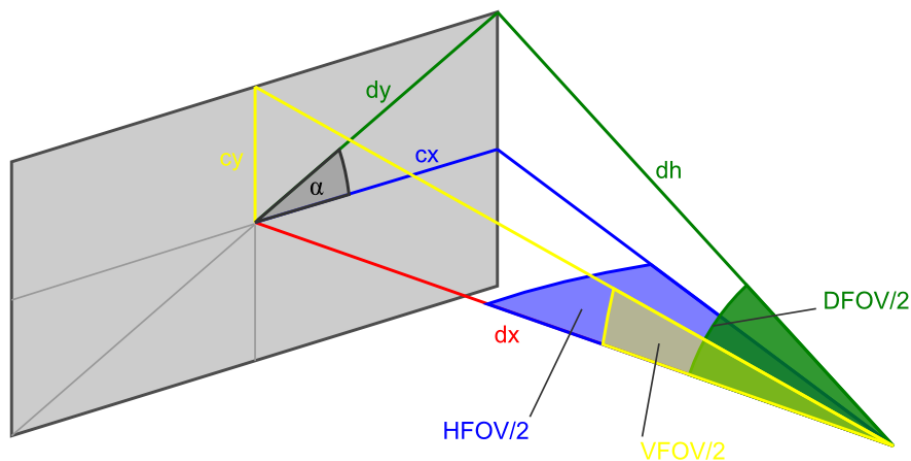


Figure 1.3: FoV 视角分布

球谐函数 (Spherical Harmonics)

球谐函数 (SH) 是一种用于近似球面函数的数学工具，在计算机图形学中广泛用于光照和渲染。

1. 基本概念

球谐函数的阶数 (Degree)

- 0 阶：常数项，表示均匀光照；
- 1 阶：线性变化，可以表示主要方向的光照；
- 2 阶：二次变化，可以表示更复杂的光照；
- 3 阶：更高次变化，可以表示更细致的光照细节。

2. 系数数量

对于 degree 为 L 的球谐函数，总系数数量为 $(L+1)^2$ ，例如当 L=3 时：总系数 = 16 个，包含 0 阶：1 个系数，1 阶：3 个系数，2 阶：5 个系数，3 阶：7 个系数。

3. 光照表示

- 用于表示每个高斯点的方向性光照响应；
- 较高的 degree 可以表示更精细的光照变化；
- 默认值 3 提供了良好的视觉质量和计算效率的平衡。

4. 性能影响

- degree 越高，需要存储的系数越多；

- 计算复杂度随 degree 增加而增加;
- 内存占用也随 degree 增加而增加.

5. 质量与效率权衡

- 较低 degree(0-1): 基础光照, 效果简单;
- 中等 degree(2-3): 较好的视觉效果, 适中的计算量;
- 高 degree(4+): 更精细的光照细节, 但计算成本高.

关键帧

关键帧 (代表帧, Key frame) 是视频中一个镜头的关键图像帧, 可以反映一个镜头的主要内容。关键帧间隔指的是两个关键帧之间间隔的中间帧数量, + 较小的间隔 = 更细腻动画 + 较大的间隔 = 更粗糙但效率更高

UV 坐标系统

UV 坐标是 3D 模型的二维纹理坐标系统, 用于定义如何将 2D 纹理图像映射到 3D 模型表面。

1. 基本概念

- U: 横向坐标 (类似 X 轴);
- V: 纵向坐标 (类似 Y 轴);
- 范围: 通常在 $[0,1]$ 或 $[-1,1]$ 之间.

2. UV 映射原理



3. UV 坐标系统特点

- 二维性: 将 3D 表面”展开”到 2D 平面;
- 连续性: 保证纹理在模型表面连续;
- 无缝性: 避免纹理接缝和扭曲.

1.3 Machine Learning

1.3.1 Contrastive Learning

1.4 Digital Human

1.4.1 Model Learning

FLAME

```
flame_param = {  
    'shape': tensor,      # 形状参数  
    'expr': tensor,      # 表情参数  
    'rotation': tensor,  # 旋转参数  
    'neck_pose': tensor, # 颈部姿态  
    'jaw_pose': tensor,  # 下巴姿态  
    'eyes_pose': tensor, # 眼睛姿态  
    'translation': tensor, # 平移参数  
    'static_offset': tensor, # 静态偏移  
    'dynamic_offset': tensor # 动态偏移  
}
```

Chapter 2

Philosophy & Insight

2.1 Research Guidelines

2.1.1 Research Purpose

“大道至简”，追求 *simple and effective* 研究。学习姚期智院士的选题三问，

1. 你的研究选题真的很重要吗？
2. 有多少人在做？
3. 为什么轮到了你来做？

关于学术品味，姚期智院士曾讲，“一个好的问题应该是漂亮的，即简洁、新颖、令人惊讶；也应该是重要的，即人们关心其答案；并且应该具有普遍吸引力，能够超越学术界限，引起广泛关注”。

- Attractive (漂亮的): Simplicity; Novelty; Surprise;
- Importance (重要的): People care what the answer is;
- Universe (普遍的): Transcends academic boundaries.

2.1.2 Global Stages

1. 不要被工程化的思维所束缚，做 A+B 的工作，着重分析本质，从源头创新。

2.1.3 Early Stage

1. 前期注重多看研究论文，精度并总结，打牢基础，做研究综述。
2. **研究综述**：有多少途径能够实现研究目的，各自是从哪些地方出发的，本质的区别是什么。
3. 对研究的议题有深入且广泛的了解，目前领域中的研究进展如何，明确 SOTA 是什么；
4. 明确有哪些子议题及技术已经被探讨，可以知道哪些还可以开发以及哪些技术可以被自己的研究议题拿来应用；

2.1.4 Research Survey

Purpose

1. 对研究的议题有深入且广泛的了解，目前领域中的研究进展如何，明确 SOTA 是什么；
2. 明确有哪些子议题及技术已经被探讨，可以知道哪些还可以开发以及哪些技术可以被自己的研究议题拿来应用；

Roadmap

1. 在该领域中主要的国际会议是什么；
2. 对这些会议近几年来（至少三年）所发表过的论文的 title 以及 abstract 浏览一次，寻找跟研究议题比较相关的论文（十来篇到二、三十篇或者更多）；
3. 阅读论文的 Introduction，做个初步的浏览，然后将这些论文依照它们与自己的研究议题的相关性做个大致的排序；
4. 依序对这些论文仔细地研读，每篇论文的重点是什么、关键的技术又是什么；
5. 文献追溯，研读的每篇论文，需要关注于它们探讨过哪些相关的文献，并且就其中与自己研究议题最相关的论文，也需要仔细地研读（追溯大概三、四代）。

Survey Stage

在这一部分，主要介绍在 survey 的过程中，分为几个阶段以及每个阶段应当把握的原则（检测 Roadmap 的指南）是什么。

1. Stage 1: 第一阶段的任务是 survey 与研究议题相关的主要国际会议，通常不超过 3 到 5 个。将与研究议题相关的最近几篇论文拿出来，看看这些论文所引用的文献有没有所不知道的，如果没有，那么比这些论文早的文献你大概都没遗漏了。如果有，就把它找出来仔细地研读。
2. Stage 2: 第二阶段的任务是 survey 更多的国际会议或 workshop，注意每篇文章的发表位置，从而找到有哪些可能相关的国际会议。仔细研读这些论文有什么突破，同时注意这些论文引用的参考文献有没有尚未读过的，没有的话表示前阶段的 survey 做的很仔细，有的话就需要把这些遗漏的论文找出来研读，并且用前面提到的方法再去追溯这篇论文三、四代以内的参考文献。

通常而言，在第二阶段找到的论文不会太多，大多数的论文在 survey 的第一阶段都会看到。

这些过程一定会收敛的，而当你发现没有新的论文被你找出来之后，你的 survey 工作大概已经做的差不多了，可以准备收工全心投入研究议题上。

进入研究阶段的时机，当你发现拿到一篇新论文时只要看完它的 Introduction 之后，你就知道这篇论文的重点及猜出它用的主要技术之后，你的功力已经提升到可以进入研究议题的阶段了。在这个阶段，如果论文的研读够深入及广泛，你往往也可以发现新的研究议题。

整个 survey 的阶段会看完几篇论文？三十篇以上是至少的，如果部分只浏览 Introduction 的部分，那可能能在五、六十篇以上。

2.2 Research Habits

2.2.1 Paper Reading

跟进前沿

关注领域内发展，要跟进最新研究成果，及时搜集最新研究论文。

- 统一的渠道：X-MOL 学术平台并保持邮件推送。
- 分散的渠道：各个期刊会议的官网。

论文略读

1. 阅读题目和整体流程图；
 2. 阅读摘要和 Introduction；
 3. 选择性阅读 Method 模块。
- 阅读原则：明确研究目标、研究问题、解决方案以及方法流程。
 - 总结原则：一段话总结 + 贡献点 + 启发思路。

论文精读

1. 先阅读题目、摘要、Introduction 和 Method；
 2. 再阅读 Related Work；
 3. 选择性阅读实验部分。
- 阅读原则：明确研究目标、研究问题、解决方案以及方法流程。
 - 总结原则：一段话总结 + 手推思路。

论文总结

1. 论文阅读后及时总结，多发掘创新点、启发点；
 2. 积累论文写作经验，可以在阅读论文中总结；
 3. 集中在一个地方整理笔记，形成研究发展体系。
- 略读选择的平台：TexStudio 汇总笔记。
 - 精读选择的平台：TexStudio 汇总笔记 + 纸质笔记。

2.2.2 Paper Writing

Abstract

1. Importance of the study
2. Gap in the existing literature
3. Objective of the study
4. Method used to conduct the study
5. Key findings of the study
6. Implications of the study

2.2.3 Experiment Exploration

1. 积累代码实现，技巧、工具库以及模板等；
 2. 多进行代码阅读，关注核心代码的编写；
 3. 关注重要的实验细节，积累调参技巧。
- 积累选择的平台：TexStudio 汇总笔记。

2.2.4 Balanced Life

1. 调控健康的生活节奏，尽量不要熬夜、不要懒觉；
2. 安排适量的运动，保证每周都能有三到五次的运动锻炼；
3. 注重健康饮食，忌高油高盐，保持均衡的营养摄入。

2.2.5 Future Planning

1. 提起规划未来半年到一年的安排；
2. 半年内的计划需清晰具象；
3. 一年内的计划可模糊笼统。

Chapter 3

Views & Inspiration

3.1 Audio-driven Methods with 3D Geometry

3.1.1 Mesh/Template-based

[TOG 2017 ADFA] [Regressive Models] [Lip Synchronization] [Expression]

Title 3.1.1 Audio-Driven Facial Animation by Joint End-to-End Learning of Pose and Emotion

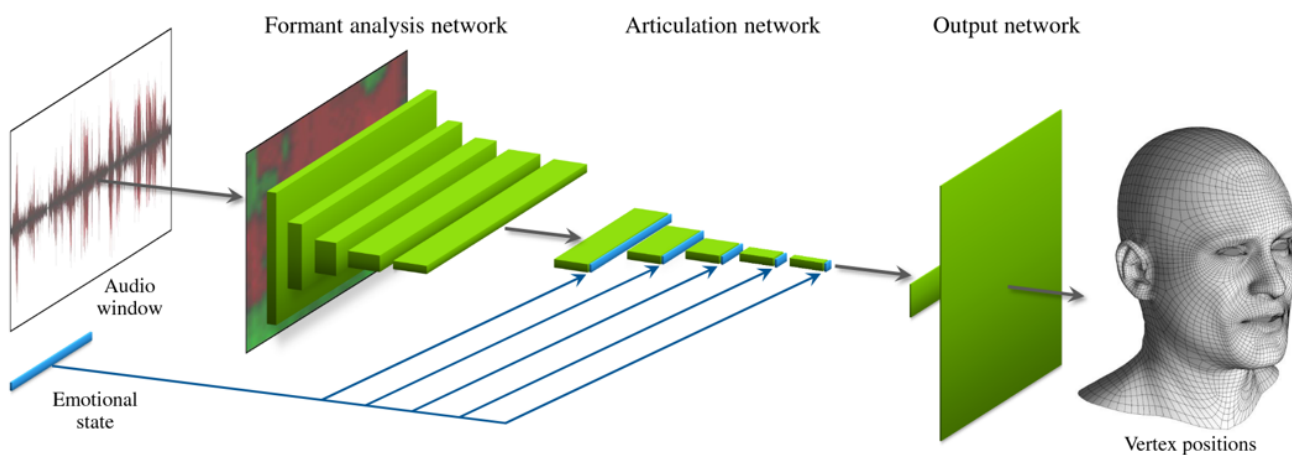


Figure 3.1: ADFA

1. Task: 利用 CNN 网络端到端地预测 Mesh 顶点位置，驱动生成 Talking Head 并引入 Emotional Vector 控制表情生成。
2. Motion: Mesh Vertex
3. Dataset: 训练目标借助商业 DI4D PRO 系统通过 9 个同步相机以 30Hz 频率进行采集获得三维 Mesh 顶点位置；训练数据对于每个 Actor 采集 3-5 分钟的音频，涵盖两个 Actors（5 分钟 1 秒与 3 分钟 45 秒）。
4. Analysis: 捕获了人物的特异性，是一个 subject specific 的方法。

[CVPR 2019 VOCA] [Regressive Models] [Lip Synchronization] [Expression]

Title 3.1.2 Capture, Learning, and Synthesis of 3D Speaking Styles

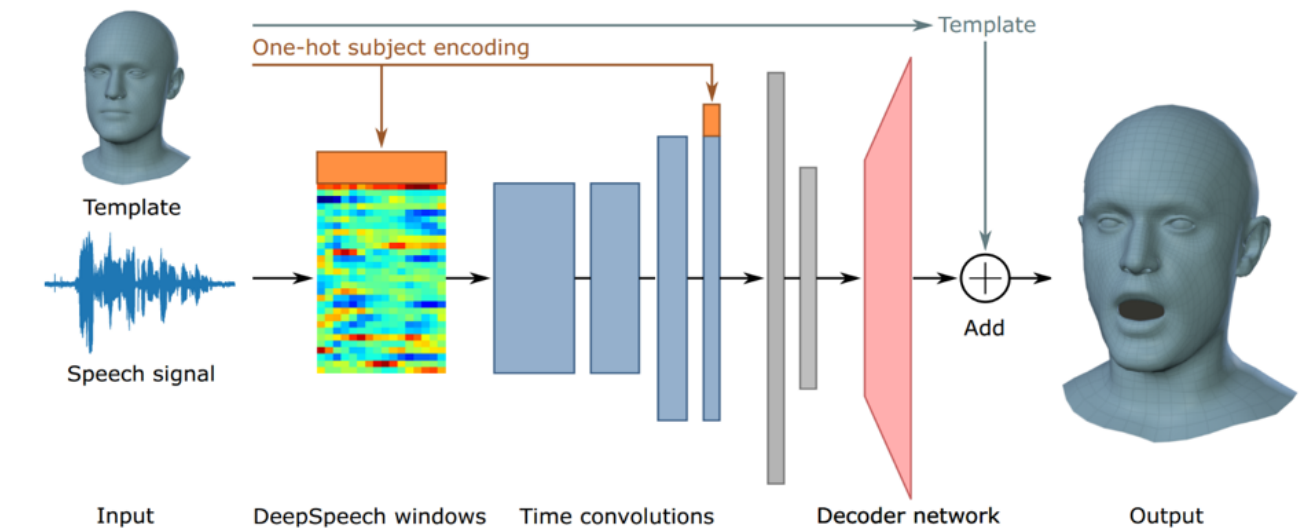


Figure 3.2: VOCA

1. Task: 基于 DNN 构建编码解码器，从音频中预测 Mesh 顶点位置偏移量，将身份从人脸运动信息中解耦分离，实现多人物之间的泛化性。
2. Motion: Mesh Vertex Offsets
3. Dataset: [VOCASET](#)，包括 6 个男性以及 6 个女性的 4D 扫描数据，每个人物有 120-200s 序列。

[CVPR 2023 CodeTalker] [Generative Models] [Lip Synchronization] [Expression]

Title 3.1.3 CodeTalker: Speech-Driven 3D Facial Animation with Discrete Motion Prior

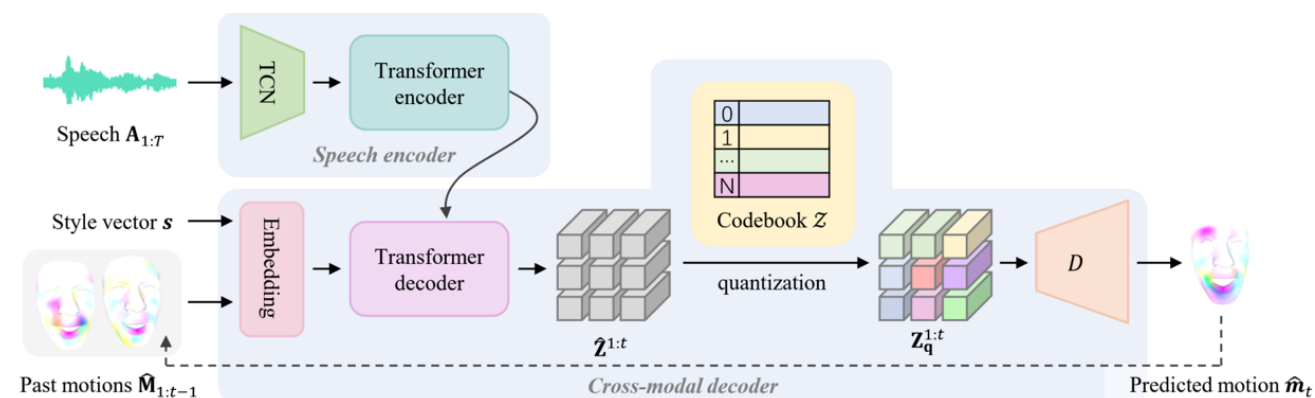


Figure 3.3: CodeTalker

1. Task: For reducing the cross-modal mapping uncertainty and regression-to-mean problem, it casts speech-driven facial animation as a code query task using the learned codebook.
2. Motivation: **By mapping the speech to the finite proxy space (a finite and discrete space)**, the uncertainty of the speech-to-motion mapping is significantly attenuated and promotes the quality of motion synthesis.
3. Motion: Mesh Vertex Offsets
4. Dataset: [BIWI](#) contains 40 unique sentences shared across all speakers in the dataset and [VOCASET](#) contains 255 unique sentences, which are partially shared among different speakers.

[ICCV 2025 GaussianSpeech] [Generative Models] [Lip Synchronization] [Expression]

Title 3.1.4 GaussianSpeech: Audio-Driven Gaussian Avatars

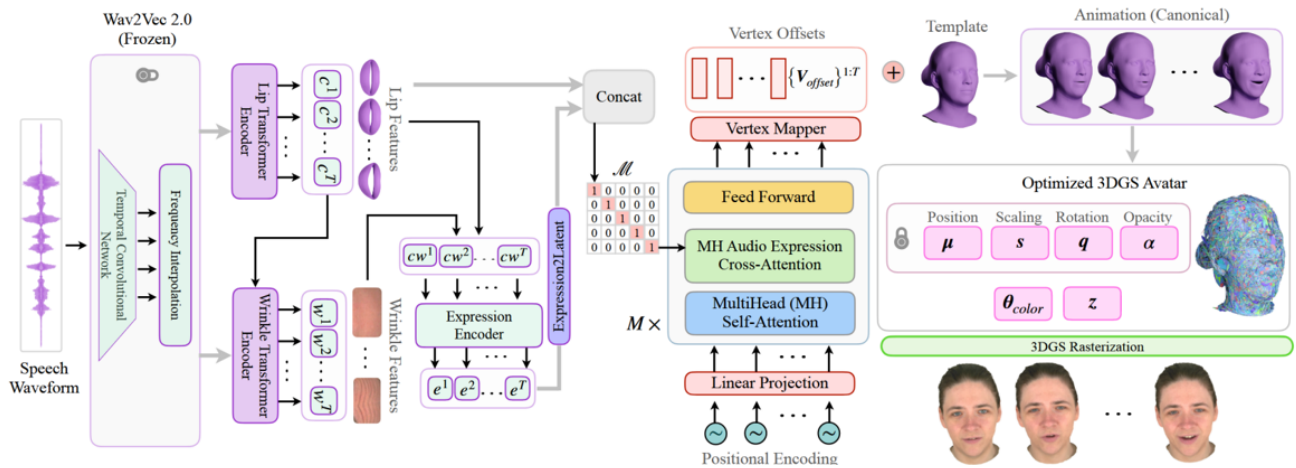


Figure 3.4: GaussianSpeech

1. Task: Decomposing lip features and winkle features from audio input to construct the related expression. Using learned expression predict the mesh vertex offsets to animate the FLAME mesh and subsequently render the results by optimized 3DGS avatar.
2. Motivation: achieving 3D consistent, free-viewpoint photorealistic synthesis, real-time rendering
3. Motion: Mesh Vertex Offsets
4. Dataset: [Multi-View Audio-Visual Dataset](#)

[IJCAI 2025 GLDiTalker] [Generative Models] [Lip Synchronization] [Expression]

Title 3.1.5 Speech-Driven 3D Facial Animation with Graph Latent Diffusion Transformer

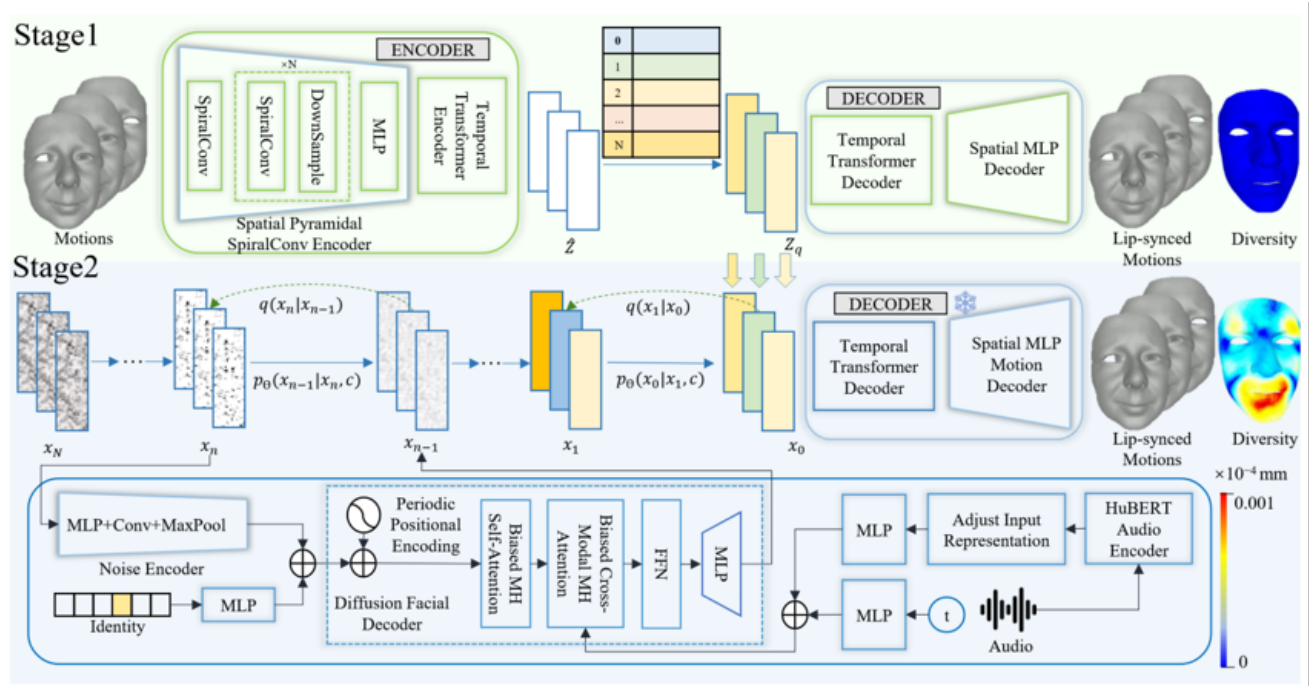


Figure 3.5: GLDiTalker

1. Task: enhance lip-sync accuracy using graph based encoder and codebook, enhance motion diversity based on latent diffusion model.
2. Motivation: modality inconsistencies, modality inconsistencies.
3. Motion: Mesh Vertex Offsets
4. Dataset: [BIWI](#) contains 40 unique sentences shared across all speakers in the dataset and [VOCASET](#) contains 255 unique sentences, which are partially shared among different speakers.

3.1.2 Keypoints

3.1.3 3DMMs

3.1.4 Hybrid Prior

3.1.5 Nerf-based

3.1.6 3DGS-based

3.2 Benchmarking

3.2.1 Datasets

3.2.2 Evaluation Metrics

PSNR

PSNR (Peak Signal-to-Noise Ratio), 即峰值信噪比, 是衡量图像质量的指标之一。基于均方误差 (MSE) 计算, 通过量化像素级误差并将其转换为分贝 (dB) 单位。

$$MSE = \frac{1}{mn} \sum_{i=0}^{m-1} \sum_{j=0}^{n-1} (I(i, j) - K(i, j))^2 \quad (3.1)$$

其中, I 为原始图像, K 为是帧图像, 图像尺寸为 $m \times n$ 。基于 MSE 可以进一步地计算出 PSNR 指标。

$$PSNR = 10 \times \log_{10} \left(\frac{MAX^2}{MSE} \right) \quad (3.2)$$

对于 8 位图像 (每个像素值的编码范围为 0 ~ 255), $MAX=255$ 。PSNR 值越高, 表示图像质量越好, 失真越小。

1. 高于 40dB: 说明图像质量极好 (即非常接近原始图像);
2. 30—40dB: 通常表示图像质量是好的 (即失真可以察觉但可以接受);
3. 20—30dB: 说明图像质量差;
4. 低于 20dB: 图像质量不可接受。

指标优势在于计算简单, 适用于多种图像处理场景的横向对比。但与人眼感知不一致, PSNR 基于像素级误差, 可能无法反映人眼对纹理、结构变化的敏感度。例如, 模糊图像可能与噪声图像 PSNR 相近, 但人眼感知差异显著。而且受限于色彩空间, 默认计算基于亮度分量 (如 YUV 的 Y 通道), 对颜色失真不敏感。

SSIM

SSIM (Structural Similarity Index Measure, 结构相似性指数) 是一种用于衡量两幅图像相似度的全参考客观评价指标, 其核心思想是通过模拟人类视觉系统 (HVS) 对亮度、对比度和结构信息的敏感度进行综合评估。

SSIM 的计算基于滑动窗口实现, 即每次计算均从图片上取一个尺寸为 $N \times N$ 的窗口, 基于窗口计算 SSIM 指标, 遍历整张图像后再将所有窗口的数值取平均值, 作为整张图像的 SSIM 指标。假设 x 表示第一张图像窗口中的数据, y 表示第二张图像窗口中的数据, 通过以下三个维度计算原始图像与失真图像的相似度。

- 亮度 (Luminance)

基于图像局部均值计算, 反映亮度差异:

$$l(x, y) = \frac{2\mu_x\mu_y + c_1}{\mu_x^2 + \mu_y^2 + c_1} \quad (3.3)$$

其中, μ_x 和 μ_y 分别表示 x 和 y 的均值, $c_1 = (k_1L)^2$ 避免分母为 0, L 为像素动态范围 (如 8 位图像 $L = 255$), k_1 为小常数 (默认 0.01)。

- 对比度相似性 (Contrast)

基于标准差计算, 衡量纹理强度差异:

$$c(x, y) = \frac{2\sigma_x\sigma_y + c_2}{\sigma_x^2 + \sigma_y^2 + c_2} \quad (3.4)$$

其中, σ_x 和 σ_y 表示 x 和 y 图像块的标准差, $c_2 = (k_2L)^2$ 避免分母为 0, L 为像素动态范围 (如 8 位图像 $L = 255$), k_2 为小常数 (默认 0.03)。

- 结构相似性 (Structure)

通过协方差衡量像素间关系, 反映边缘和纹理的位置一致性:

$$s(x, y) = \frac{\sigma_{xy} + c_3}{\sigma_x\sigma_y + c_3} \quad (3.5)$$

其中, σ_{xy} 表示 x 和 y 之间的协方差, 且 $c_3 = c_2/2$ 。

最终, 综合以上三项得到最后的计算指标,

$$SSIM(x, y) = l(x, y)^\alpha \cdot c(x, y)^\beta \cdot s(x, y)^\gamma \quad (3.6)$$

如果, $\alpha = \beta = \gamma = 1$, 则常用的 SSIM 公式为,

$$SSIM(x, y) = \frac{(2\mu_x\mu_y + c_1)(2\sigma_{xy} + c_2)}{(\mu_x^2 + \mu_y^2 + c_1)(\sigma_x^2 + \sigma_y^2 + c_2)} \quad (3.7)$$

LPIPS

LPIPS (Learned Perceptual Image Patch Similarity) 是一种基于深度学习的图像相似性度量指标, 旨在更贴近人类视觉感知的图像质量评估方法。与传统指标 (如 PSNR、SSIM) 不同, LPIPS 通过预训练的神经网络提取图像的高层语义特征, 计算两幅图像在特征空间中的差异, 从而更准确地反映人类对图像差异的主观感知。

LPIPS 比传统方法 (比如 L2/PSNR, SSIM, FSIM) 更符合人类的感知情况。LPIPS 的值越低表示两张图像越相似, 反之, 则差异越大。具有以下的特点,

1. 感知优先: 关注语义级差异 (如物体结构、纹理), 而非像素级误差。
2. 鲁棒性: 对平移、旋转、亮度变化不敏感, 但对语义失真敏感 (如模糊、扭曲)。

假设两个输入经过 l 层处理并进行激活和归一化后的输出为, $\hat{y}_l^l, \hat{y}_0^l \in \mathcal{R}^{H_l \times W_l \times C_l}$, 再经过权重点乘后计算 L2 距离, 最后在空间上取图像尺寸的平均值并逐通道求和, 以获得最后的指标值。

$$d(x, x_0) = \sum_l \frac{1}{H_l W_l} \sum_{h, w} \|w_l \odot (\hat{y}_{hw}^l - \hat{y}_{0hw}^l)\|_2^2 \quad (3.8)$$

Chapter 4

Coding & Skills

4.1 Environment Configuration

4.1.1 Miniconda Installation On Linux

```
# Fetch the miniconda script
export HOME=$PWD
wget -q https://repo.anaconda.com/miniconda/ \
Miniconda3-py37_4.12.0-Linux-x86_64.sh -O miniconda.sh
sh miniconda.sh -b -p $HOME/miniconda3
rm miniconda.sh
export PATH=$HOME/miniconda3/bin:$PATH

# Initialize conda
source $HOME/miniconda3/etc/profile.d/conda.sh
hash -r
conda config --set always_yes yes --set changeps1 yes

# Create new environment
conda create -n my_env python=3.8
conda activate my_env
```

4.1.2 Pytorch3d Installation on Windows

Windows 下 3D Vision 环境配置 (GaussianAvatars)

4.1.3 Ninja Installation on Linux

```
apt install ninja-build
```

4.2 Python Debugging

4.2.1 Visual Studio Code Debugging Configuration

为了能够在 Visual Studio Code 中调试 Python 项目，首先需要安装 Python 和 Python Debugger 扩展。然后如图4.1所示，点击左侧的调试按钮，添加 lanch.json 文件，并根据代码的配置示例创建项目配置。

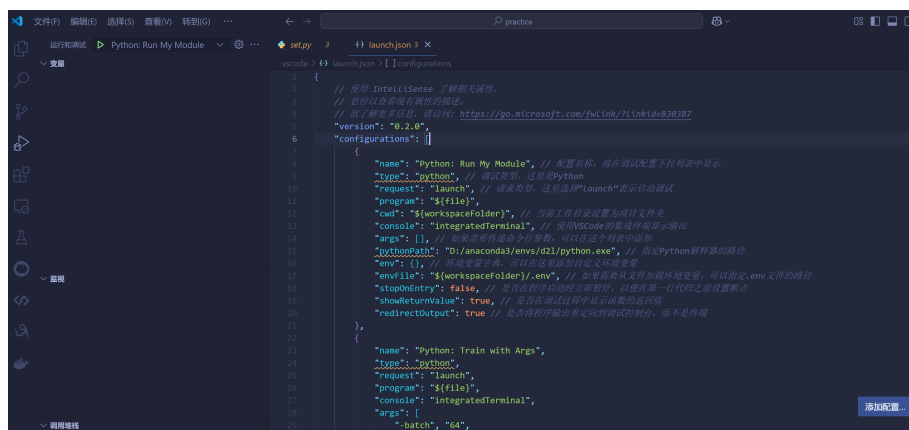


Figure 4.1: Python 项目调试配置

```
{
  "version": "0.2.0",
  "configurations": [
    {
      // 配置名称，将在调试配置下拉列表中显示
      "name": "Python: Run My Module",
      "type": "python", // 调试类型，这里是 Python
      "request": "launch", // 请求类型，这里选择 "launch" 表示启动调试
      "program": "${file}",
      "cwd": "${workspaceFolder}", // 当前工作目录设置为项目文件夹
      // 使用 VSCode 的集成终端显示输出
      "console": "integratedTerminal",
      "args": [
        "--batch", "64",
        "--epoches", "30",
      ], // 如果需要传递命令行参数，可以在这个列表中添加
      // 指定 Python 解释器的路径
    }
  ]
}
```

```

"pythonPath": "D:/anaconda3/envs/d2l/python.exe",
"env": {}, // 环境变量字典，可以在这里添加自定义环境变量
// 如果需要从文件加载环境变量，可以指定.env 文件的路径
"envFile": "${workspaceFolder}/.env",
// 是否在程序启动时立即暂停，以便在第一行代码之前设置断点
"stopOnEntry": false,
"showReturnValue": true, // 是否在调试过程中显示函数的返回值
"redirectOutput": true // 是否将程序输出重定向到调试控制台，而不是终端
}
}
}

```

在完成以上准备工作后，设置程序断点，然后直接点击调试三角形按钮，即可开始调试，如图 4.2 所示。

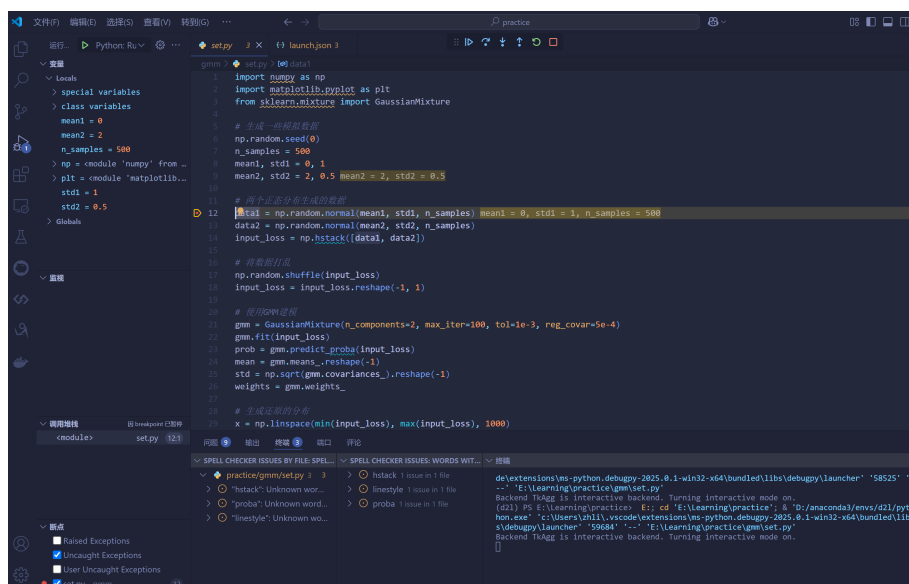


Figure 4.2: Python 程序调试示例

【内嵌调试终端找不到环境变量解决办法】

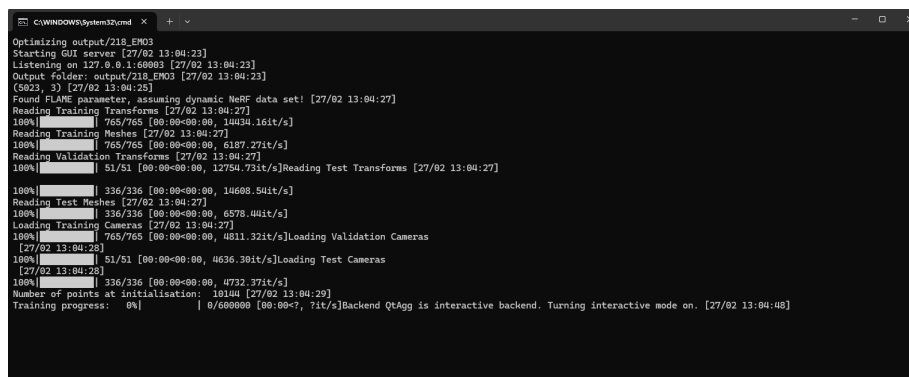


Figure 4.3: 弹出的外部终端

在一些情况下, VS Code 中调试程序无法找到环境变量, 但在 CMD 终端中可以成功运行。这样的情况, 解决办法也很简单, 将调试的终端更改为外部终端即可, 即配置 "console": "externalTerminal" 参数。配置完成后, 重新调试程序, 会调用外部的终端作为调试终端, 如图4.3所示。

4.3 Bugs with Solution

4.3.1 Version Conflict

1.
 - Q: Numpy 版本和 Pytorch 版本不兼容。
 - A: 尝试更换 Numpy 版本。

```
assert (self.faces==torch.from_numpy(flame_model.f.astype( \
'int64'))).all()
RuntimeError: Numpy is not available
```

2.
 - Q: PIL 版本更新问题。
 - A: 在 pillow 的 10.0.0 版本中, ANTIALIAS 方法被删除了, 通过命令 `pip install Pillow==9.5.0` 安装旧版本即可。

```
AttributeError: module 'PIL.Image' has no attribute 'ANTIALIAS'
```

3.
 - Q: PyAudio 安装问题。
 - A: PyAudio 官方仅支持部分的 Python 版本, 如果需要安装对应 Python=3.9 的版本, 需要从非官方安装包进行安装——[清华镜像网站](#)。

```
LibMambaUnsatisfiableError: Encountered problems while solving:
- package pyaudio-0.2.11-py27h0c8e037_1 requires python >=2.7, \
<2.8.0a0, but none of the providers can be installed
```

```
Could not solve for environment specs
The following packages are incompatible
└─ pin-1 is installable and it requires
  ┌─ python 3.9.* , which can be installed;
  └─ pyaudio is not installable because there are no viable options
```

4.
 - Q: FFmpeg 安装问题

```
Unrecognized option 'crf'.
Error splitting the argument list: Option not found.
```


- A: 所安装的 FFmpeg 不是完整版本, 尤其是当 FFmpeg 是从源码编译而来时, 默认不编译 lib264 组件。在这个地方我是在 Conda 虚拟环境中直接安装的 FFmpeg, 应当首先卸载虚拟环境中的 FFmpeg, 然后在 Linux 终端直接安装完整的 FFmpeg。

```
sudo apt install ffmpeg
```

5.
 - Q: torch 未知报错

```
D:\00MyWorkplace\01anaconda\envs\RAD-NeRF\lib\site-packages \
\torch\include\pybind11\cast.h(1429): \
error: too few arguments for template template parameter "Tuple"
```

- A: 将 cast.h 中报错信息相应位置附近 (具体在 1506-1507 行), 注释掉以下内容。

```
// template <typename T1, typename T2> class type_caster<std::pair<T1, T2>>
//      : public tuple_caster<std::pair, T1, T2> {};

template <typename... Ts> class type_caster<std::tuple<Ts...>>
    : public tuple_caster<std::tuple, Ts...> {};
```

4.4 Coding Tricks

4.4.1 Python

pathlib

pathlib 是一个专用于路径管理的代码库, 功能比 os.path 更加灵活。

- 类型指定: `input: Path`
- 路径赋值: `'../../dir'`
- 判断路径是否存在: `Path.exists()`
- 获取父目录: `Path.parent`
- 通配符匹配: `Path.glob()`

8_000

Python 中数字表示含有下划线时, 仅用于提高大数字的可读性, 对数值没有影响。

parser

在解析终端输入的指定命令行参数时非常用借鉴意义, 如下所示。

```
args = parser.parse_args(sys.argv[1:])
```

其中, `sys.argv[0]` 是 Python 脚本名称, `sys.argv[1]` 包含所有实际的命令行参数。例如, 启动命令为, `python train.py --batch-size 32 --learning-rate 0.001` 对应, `sys.argv[0]` 表示 `train.py`, `sys.argv[1]` 表示 `['--batch-size', '32', '--learning-rate', '0.0']`。

with

`with open` 同时打开多个文件的写法值得参考。

```
with open(scene_info.ply_path, 'rb') as src_file, \
    open(os.path.join(self.model_path, "input.ply"), 'wb') \
        as dest_file:
    dest_file.write(src_file.read())
```

tensor.scatter_add_

`scatter_add_` 是 PyTorch 中的一个张量操作函数, 用于将一个张量中的值按照指定的索引累加到另一个张量中。

`torch.Tensor.scatter_add_(dim, index, src) -> Tensor`

`scatter_add_` 会根据 `index` 中的索引, 将 `src` 中的值累加到目标张量 (即调用该方法的张量) 的指定位置。

下面代码实现了绑定计数的功能, 根据绑定关系 (下标索引) 将全 1 向量统计到每个面中。

```
# 创建计数器和绑定关系
counter = torch.zeros(5, device="cuda") # 假设有 5 个面
# 4 个点绑定到不同的面
binding = torch.tensor([0, 2, 2, 1], device="cuda")
# 使用 scatter_add_ 统计每个面绑定的点数
counter.scatter_add_(0, binding,
    torch.ones_like(binding, dtype=counter.dtype, device="cuda"))

print(counter) # 输出: tensor([1, 1, 2, 0, 0])
```

register_buffer

`register_buffer` 是 PyTorch 中 `nn.Module` 的一个方法, 用于注册不需要计算梯度的张量。

```
def register_buffer(self, name: str, tensor: Optional[Tensor], \
    persistent: bool = True)
```

`register_buffer` 的这些数据是模型状态的一部分, 但不需要梯度; 在模型移动到 GPU 时会自动转移, 会被保存在模型的 `state_dict` 中。

通过注册缓冲变量后，可以当作属性直接访问使用：

```
self.register_buffer(
    "v_template",
    to_tensor(to_np(flame_model.v_template), dtype=self.dtype)
)
# 1. 直接访问
vertices = self.v_template

# 2. 获取所有 buffers
for name, buffer in self.named_buffers():
    print(f"Buffer {name}: {buffer.shape}")
```

最后总结一下，torch.nn 库中 Parameter 和 Buffer 的区别：

- Parameter: 需要梯度计算，会被优化器更新，用于模型权重。
- Buffer: 不需要梯度，不会被优化器更新，用于常量或中间状态。

@dataclass

@dataclass 是 Python 3.7 引入的一个装饰器，用于自动生成数据类的特殊方法，比如 `__init__`、`__repr__`、`__eq__` 等，常用作配置类、数据传输对象以及不可变数据结构。

```
from dataclasses import dataclass

# 不使用 @dataclass
class PersonWithoutDecorator:
    def __init__(self, name: str, age: int):
        self.name = name
        self.age = age

    def __repr__(self):
        """
        用于生成字符串表示，主要控制输出格式。
        """
        return f"Person(name='{self.name}', age={self.age})"

    def __eq__(self, other):
        """
        实现相等比较，重载 == 运算符。
        """
        return self.name == other.name and self.age == other.age
```

```
# 使用 @dataclass - 自动生成上述所有方法
@dataclass
class Person:
    name: str
    age: int

@dataclass(frozen=True) # frozen=True 使对象不可变
class Point3D:
    x: float
    y: float
    z: float

PWD = PersonWithoutDecorator("Zhihao Li", 22)
PWDOld = PersonWithoutDecorator("Zhihao Li", 23)
Per = Person("Zhihao Li", 22)
point = Point3D(1, 2, 3)

# 判断比较以及装饰器
print(PWD)
print(PWD == PWDOld)
print(Per)

# 判断不可变数据结构
print(point)
try:
    point.x = 3
    print("successfully modified the data")
    print(point)
except:
    print("not support modifying the data")
    print(point)

# results
Person(name='Zhihao Li', age=22)
False
Person(name='Zhihao Li', age=22)
Point3D(x=1, y=2, z=3)
not support modifying the data
Point3D(x=1, y=2, z=3)
```

field

```
class Config(Mini3DViewerConfig):
    pipeline: PipelineConfig = field(default_factory=PipelineConfig)
```

从上面可以分析得出, PipelineConfig 是一个类, 每个 Config 实例都需要一个独立的 PipelineConfig 实例, 如果直接写 pipeline: PipelineConfig = PipelineConfig(), 所有 Config 实例会共享同一个 PipelineConfig 实例。因此 field 函数主要用于在初始化中生成动态的实例或数据, 常见于类初始化中。

```
from dataclasses import field

field(
    default_factory=PipelineConfig,  # 用于生成默认值的工厂函数
    init=True,                      # 是否作为 __init__ 参数
    repr=True,                      # 是否包含在 repr 中
    compare=True,                   # 是否参与比较
    hash=None,                      # 是否参与哈希计算
    metadata=None                   # 字段元数据
)
```

```
from dataclasses import dataclass, field

# 可变默认值
@dataclass
class Example:
    list_field: list = field(default_factory=list)
    dict_field: dict = field(default_factory=dict)

exp1 = Example([1,2], {"1":3})
exp2 = Example([2,2], {"1":3, "2":3})
print(exp1)
print(exp2)

exp1.list_field.append(3)
print(exp1)

# 定义对象实例
class Number:
    def __init__(self):
        self.number = 0
```

```

@dataclass
class Config:
    number: Number = field(default_factory=Number)

cfg1 = Config()
cfg2 = Config()
cfg1.number.number = 1
cfg2.number.number = 2
print(cfg1.number.number)
print(cfg2.number.number)

# results
Example(list_field=[1, 2], dict_field={'1': 3})
Example(list_field=[2, 2], dict_field={'1': 3, '2': 3})
Example(list_field=[1, 2, 3], dict_field={'1': 3})
1
2

```

tyro.cli

`tyro.cli()` 是一个用于将 `dataclass` 转换为命令行参数的工具。

预先定义的 `dataclass` 后，通过语句 `cfg = tyro.cli(Config)` 将其转换为支持命令行参数的配置，之后便可以在命令后中指定对应的参数，作用等同于 `argparse`。

```

@dataclass
class Config(Mini3DViewerConfig):
    pipeline: PipelineConfig = field(default_factory=PipelineConfig)
    cam_convention: Literal["opengl", "opencv"] = "opencv"
    point_path: Optional[Path] = None

python local_viewer.py \
    --point_path path/to/point_cloud.ply \
    --cam_convention opencv \
    --sh_degree 3 \
    --fps 25

```

setattr

`setattr` 主要用于设置对象的实例属性，同理 `hasattr` 主要用于判断对象是否具有某实例属性。

```

class Struct(object):
    def __init__(self, **kwargs):
        for key, val in kwargs.items():
            setattr(self, key, val)

# 创建实例并设置属性
config = Struct(name="FLAME", version="1.0", features=["shape",\
               "expression"])

# 访问属性
print(config.name)          # 输出: "FLAME"
print(config.version)       # 输出: "1.0"
print(config.features)      # 输出: ["shape", "expression"]

# 单独使用该方法设置对象 config 的属性
setattr(config, "test", "val")
print(config.test)

# results
FLAME
1.0
['shape', 'expression']
val

```

torch.autograd.set_detect_anomaly

PyTorch 中的一个函数，用于在自动求导过程中检测和调试潜在的计算问题。它的主要作用是帮助开发者定位梯度计算中的异常，例如 NaN 或 Inf 值，主要用于调试代码。

```
torch.autograd.set_detect_anomaly(args.detect_anomaly)
```

pprint

对于 Parser 的 Config 能够很好的打印输出。

```
import pprint
pprint.pprint(vars(args))
```

pathlib

pathlib.Path 是 Python 3 推荐的路径处理方式，它会自动处理不同操作系统的路径分隔符，提供了更多有用的路径操作方法。

```
from pathlib import Path
```

将字符串路径转换为统一文件路径，并通过 `parts` 属性进行分割

```
src = Path(image_path).parts[-3]
```

4.4.2 Cmd On Windows

pwd

打印当前目录路径

```
PS D:\ZHaoLi\02.Hobbies\01.Blog> pwd
```

```
Path
```

```
----
```

```
D:\ZHaoLi\02.Hobbies\01.Blog
```

taskkill

在 Windows 平台上，可以通过该命令来 kill 掉指定的进程。

```
taskkill /PID 44664 /F
```

其中，/PID 指定要杀掉的进程 ID，/F 表示强制结束进程。

4.4.3 Linux Usage

whereis

在 Linux/Unix 系统下，用于定位可执行文件、源代码文件及 man 手册路径，快速查找指定命令或程序的相关文件位置。

`whereis` [选项] 文件名

- -b 只查找可执行文件 (binary)。
- -m 只查找 man 手册路径。
- -s 只查找源代码路径。
- -u 只显示缺少某类文件的条目 (例如缺少 man 页)。

```
whereis ls
```

```
# 输出: ls: /bin/ls /usr/share/man/man1/ls.1.gz
```

which

只查找命令的可执行文件位置，按照 PATH 环境变量顺序返回第一个匹配的路径。

`which` [选项] 命令名

```
which ls
```

```
# 输出: /bin/ls
```


Chapter 5

Grammar & Writings

5.1 Mathematical Symbols in LaTeX

5.1.1 集合及向量空间的表示

Rendered Symbol	Source Code	Rendered Symbol	Source Code
S	<code>$\mathcal{S}$</code>	$\mathbb{R}$	<code>$\mathbb{R}$</code>

Table 5.1: LaTeX Symbols and Their Source Code

5.1.2 运算符号

Rendered Symbol	Source Code	Rendered Symbol	Source Code
$\ll$	<code>$\ll$</code>	$\mathbb{R}$	<code>$\mathbb{R}$</code>

Table 5.2: LaTeX Symbols and Their Source Code

5.1.3 多行公式

在 Latex 中编写多行公式, 一般采用 `\begin{cases} \dots \end{cases}` 和 `\begin{dcases} \dots \end{dcases}` 表示分段函数, 并且它们的主要区别在于公式的排版方式:

1. `{cases}`: 采用普通的数学样式 (textstyle), 适用于行内或小型公式。生成的分段函数会比较紧凑, 分式等数学表达式会缩小。

```
\begin{align}
l_0(x) =
\begin{cases}
\frac{(x - x_1)}{x_0 - x_1}, x \in [x_0, x_1] \\
\end{cases}
```

```

l_i(x)=
\begin{cases}
\frac{x-x_{i-1}}{x_0-x_1}, x \in [x_0, x_1], \\
0, \text{ 其他}
\end{cases}
\end{align}

```

$$l_0(x) = \begin{cases} \frac{x-x_1}{x_0-x_1}, x \in [x_0, x_1] \\ 0, \text{ 其他} \end{cases} \quad l_i(x) = \begin{cases} \frac{x-x_{i-1}}{x_i-x_{i+1}}, x \in [x_i, x_{i+1}] \\ 0, \end{cases} \quad (5.1)$$

2. {dcases}: 采用 display 数学样式 (displaystyle), 适用于更清晰的公式排版。适用于较大的数学符号 (如分式、求和符号等), 使其在每一行都保持较大的尺寸。

```

\begin{align}
& l_0(x) = \begin{dcases}
\frac{(x - x_1)}{x_0 - x_1}, & x \in [x_0, x_1] \\
0, & \text{其他}
\end{dcases} \\
& l_i(x) = \begin{dcases}
\frac{x-x_{i-1}}{x_i-x_{i+1}}, & x \in [x_{i-1}, x_i] \\
\frac{x-x_{i+1}}{x_i-x_{i+1}}, & x \in [x_i, x_{i+1}] \\
0, & \text{其他}
\end{dcases} \quad i = 1, 2, \dots, n-1,
\end{align}

```

$$l_0(x) = \begin{cases} \frac{(x - x_1)}{x_0 - x_1}, & x \in [x_0, x_1] \\ 0, & \text{其他} \end{cases} \quad (5.2)$$

$$l_i(x) = \begin{cases} \frac{x - x_{i-1}}{x_i - x_{i-1}}, & x \in [x_{i-1}, x_i], \\ \frac{x - x_{i+1}}{x_i - x_{i+1}}, & x \in [x_i, x_{i+1}] \\ 0, & \text{其他} \end{cases} \quad i = 1, 2, \dots, n-1, \quad (5.3)$$

5.2 Extended Markdown

在 Markdown 文件中实现下拉式列表, 可以通过 'html' 语言实现, 具体语法如下:

```

<details>
  <summary> Dependencies (click to expand) </summary>

```

```
## Dependencies
```

- PyTorch 1.4
- matplotlib
- numpy
- imageio
- imageio-ffmpeg
- configargparse

The LLFF data loader requires ImageMagick.

</details>

5.3 Elaborating Ideas

1. 首先阐述问题的必要性，然后引出自己的方法。

Chapter 6

Templates & Tools

6.1 Templates

6.1.1 Loss Calculation

L1 Loss

L1 损失也称为平均绝对误差 (MAE)，计算公式如6.1。对异常值不敏感，适合希望减少大误差影响的场景；在零点不可导，优化时需使用次梯度（如符号函数）。

$$L1_{loss} = \frac{1}{N} \sum_{i=1}^N |y_{pred}^i - y_{true}^i| \quad (6.1)$$

```
def l1_loss(pred, gt):  
    return torch.abs((pred - gt)).mean()
```

在 Pytorch 中，一般通过 `torch.nn` 实现。

```
import torch  
import torch.nn as nn  
  
# 默认是 'mean'（平均），也可设为 'sum'（求和）或 'none'（保留逐元素误差）  
criterion = nn.L1Loss(reduction='mean')  
loss = criterion(y_pred, y_true)
```

L2 范数

L2 范数是数学和机器学习中广泛使用的一种“向量长度”或“距离度量”工具，本质上是计算向量元素的平方和的平方根。对于一个向量而言，其 L2 范数表示为公式6.2所示。

$$\|x\|_2 = \sqrt{x_1^2 + x_2^2 + \cdots + x_n^2} \quad (6.2)$$

对于论文中的公式6.3，对于向量范数加入了阈值截断，其中 μ 表示高斯球的中心坐标， ϵ 表示设定的阈值。

$$L_{pos} = \max(\|\mu\|_2 - \epsilon_{pos}, 0) \quad (6.3)$$

在代码实现中，对于最值阶段的操作选取了 F.relu 激活函数实现，为了约束高四点在局部坐标系的位置不能偏差坐标系原点太大，先计算局部高斯点坐标的范数，然后减去阈值，并通过激活函数判断是否超过阈值，保证在阈值范围内的高斯点是合理的。

```
F.relu(gaussians._xyz[visibility_filter].norm(dim=1) - \
      opt.threshold_xyz).mean()
```

L3 范数

L3 范数是 Lp 范数的一种 (p=3)，表示向量元素绝对值的三次方之和的三次方根。它在数学上有明确的定义，但在实际应用（如机器学习、优化）中并不常见。对于一个向量而言，其 L3 范数表示为公式6.4所示。

$$\|x\|_3 = (x_1^3 + x_2^3 + \cdots + x_n^3)^{1/3} \quad (6.4)$$

Scaling Loss

在控制高斯球在 x, y, z 三个方向上的缩放时，应当调整 L2 Norm 的顺序，如公式6.5所示。

$$L_{scaling} = \|\max(s - \epsilon_{scaling}, 0)\|_2 \quad (6.5)$$

在代码中实现如下，通过指数函数将缩放系数进一步放大，然后对于三个方向的缩放先进行阈值判断，通过激活函数滤除阈值范围内的情况，最后再计算 L2 范数作为损失项。

```
F.relu(torch.exp(gaussians._scaling[visibility_filter]) - \
      opt.threshold_scale).norm(dim=1).mean()
```

Cosine Similarity Loss

给定两个向量 $\vec{u}, \vec{v}$ ，希望计算两个向量的夹角余弦值以此衡量之间的相似度，只关注向量的方向而忽视向量的大小关系，对长度不敏感，适合高维稀疏数据。

$$\text{Cos} \langle \vec{u}, \vec{v} \rangle = \frac{\vec{u} \cdot \vec{v}}{\|\vec{u}\| \|\vec{v}\|} \quad (6.6)$$

```
class CosineSimiLoss(nn.Module):
    def __init__(self, batch_size):
        super(CosineSimiLoss, self).__init__()
        self.ones = Variable(torch.ones(batch_size)).cuda(async=True)
        self.l1_loss = nn.L1Loss()
```

```
def forward(self, input1, input2):
    cos_simi = F.cosine_similarity(input1, input2, eps=1e-5)
    return self.l1_loss(cos_simi, self.ones)
```

Pixel-level Reconstruction Loss

Pixel-level Reconstruction Loss（像素级重建损失）用于衡量生成图像与原始图像在像素级别上的差异，确保生成图像在像素级别上尽可能接近原始输入，推动模型学习细节特征，常见的方式有 MSE 和 MAE。

优点是计算简单高效，直接约束像素匹配。缺点是可能导致生成结果过于平滑（丢失高频细节），例如合成的视频帧流畅度较低，需配合其他损失（如 SSIM、感知损失）提升视觉效果。

1. 均方误差（MSE, Mean Squared Error）

$$MSE = \frac{1}{N} \sum_{i=1}^N (x_i - \hat{x}_i)^2 \quad (6.7)$$

N 表示像素总数，通过图像通道、高度与宽度的乘积计算。

2. 平均绝对误差（MAE, Mean Absolute Error）

$$MAE = \frac{1}{N} \sum_{i=1}^N |x_i - \hat{x}_i| \quad (6.8)$$

```
def pixel_mse_loss_np(original: np.ndarray, reconstructed: np.ndarray):
    """MSE"""
    return np.mean((original - reconstructed) ** 2)

def pixel_mae_loss_np(original: np.ndarray, reconstructed: np.ndarray):
    """MAE"""
    return np.mean(np.abs(original - reconstructed))
```

Perceptual Loss

Perceptual Loss 的核心思想是通过预训练的卷积神经网络（如 VGG19）提取图像的中间层特征，比较生成图像与真实图像在特征空间的差异。损失函数通常是各层特征的 均方误差（MSE）的加权和。

优点具有感知一致性：确保生成图像在结构、纹理等高层特征上与真实图像接近，而非仅像素级相似。提升生成质量：常用于 GANs、图像修复（如 Inpainting）、风格迁移等任务，减少生成模糊或失真。缺点是，权重调节困难，且忽略底层细节。

$$L_{perceptual} = \sum_i w_i \cdot \|F_i(G) - F_i(R)\|^2 \quad (6.9)$$

其中， F_i 表示 VGG19 中第 i 层的输出特征， w_i 表示可调节的权重，而 G, R 分别表示生成和真实图像。

```

import torch
import torch.nn as nn
import torchvision.models as models
from torchvision import transforms

class PerceptualLoss(nn.Module):
    def __init__(self, weights=None):
        super().__init__()
        # 加载 VGG19 的特征提取层（去掉全连接层）
        self.vgg = models.vgg19(pretrained=True).features.eval()
        self.weights = weights or [1.0, 1e-4, 1e-9, 1e-13, 1e-16]

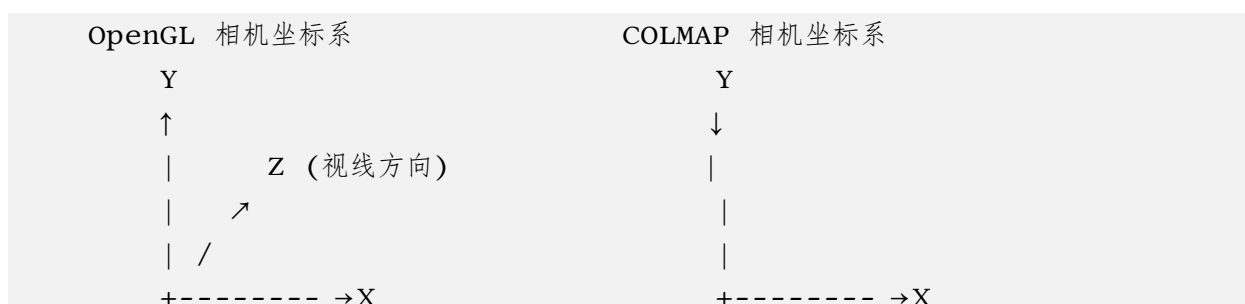
        # 输入归一化参数（与 VGG 预训练一致）
        self.mean = [0.485, 0.456, 0.406]
        self.std = [0.229, 0.224, 0.225]
        self.transform = transforms.Compose([
            transforms.Resize((224, 224)),
            transforms.Normalize(mean=self.mean, std=self.std)
        ])

    def forward(self, generated, real):
        gen = self.transform(generated)
        real = self.transform(real)
        loss = 0.0
        for layer, w in zip(self.vgg, self.weights):
            gen_feat = layer(gen)
            real_feat = layer(real)
            loss += w * nn.functional.mse_loss(gen_feat, real_feat)
        return loss

```

6.1.2 Computer Graphics

1. OpenGL/Blender camera axes(Y up, Z back) to COLMAP(Y down, z forward)



/

↙

Z (视线方向)

```
# NeRF 'transform_matrix' is a camera-to-world transform
c2w = np.array(frame["transform_matrix"])
c2w[:3, 1:3] *= -1

# get the world-to-camera transform and set R, T
w2c = np.linalg.inv(c2w)
# R is stored transposed due to 'glm' in CUDA code
R = np.transpose(w2c[:3, :3])
T = w2c[:3, 3]
```

2. 视场角与焦距的转换

根据前面推导出的公式1.8, 即可推导出相互转换关系。

```
def fov2focal(fov, pixels):
    return pixels / (2 * math.tan(fov / 2))

def focal2fov(focal, pixels):
    return 2 * math.atan(pixels / (2 * focal))
```

6.1.3 Setup Random Seed

```
def setup_seed(seed):
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)
    np.random.seed(seed) # 控制 numpy 的随机性
    random.seed(seed) # 控制 Python 内置 random 模块的随机性
    torch.backends.cudnn.deterministic = True # 让 cudnn 使用确定性算法
```

6.1.4 Load and Check Model

```
# manually load state dict
model_dict = torch.load(opt.ckpt, map_location='cpu')['model']
```



```

missing_keys, unexpected_keys = model.load_state_dict(model_dict, strict=False)

if len(missing_keys) > 0:
    print(f"[WARN] missing keys: {missing_keys}")
if len(unexpected_keys) > 0:
    print(f"[WARN] unexpected keys: {unexpected_keys}")

# freeze these keys
for k, v in model.named_parameters():
    if k in model_dict:
        print(f'[INFO] freeze {k}, {v.shape}')
        v.requires_grad = False

```

1. torch.manual_seed(seed)

设置 CPU 和当前 GPU 设备（如果有 GPU）上 PyTorch 的随机数生成器（RNG）的种子。PyTorch 在 CPU 上执行的操作，如 torch.randn(), torch.rand(), torch.randint() 等；在当前活跃的 GPU（current device）上执行的 CUDA 相关的随机数操作（如 torch.cuda.FloatTensor().normal_()）。

2. torch.cuda.manual_seed_all(seed)

为所有可见的 GPU 设备上的 CUDA 随机数生成器设置相同的种子。PyTorch 在所有 GPU 设备上执行的随机数生成，如多 GPU 训练或手动跨 GPU 分配时，确保每个 GPU 的随机状态一致；常用于多 GPU/分布式训练时。

6.2 Tools

6.2.1 FFmpeg

通过调整 FFmpeg 编码参数提升合成视频质量，优化前的命令为，

```

os.system(f"ffmpeg -y -framerate 25 -f image2 -pattern_type glob -i \
'{render_path}/*.png' -pix_fmt yuv420p {iter_path}/renders.mp4")

```

经过以下的核心优化步骤后，命令改为：

```

os.system(
    # 核心质量参数
    f"ffmpeg -y -framerate 25 -f image2 -pattern_type glob "
    f"-i '{render_path}/*.png' -c:v libx264 -crf 18 -preset slow "
    f"-tune animation -x264-params keyint=60:scenecut=0 "
    # 分辨率缩放 + 像素格式
    f"-vf \"scale=1920:-2:flags=lanczos,format=yuv420p\" "
)

```

```
f"-movflags +faststart {iter_path}/renders.mp4" # 兼容性优化
)
```

编码器选择

- 参数: `-c:v libx264`
- 作用: 采用 H.264 编码器平衡质量与压缩效率
- 原理: 相比默认编码器, libx264 支持 CRF 质量调节和高级压缩算法

质量因子调节

- 参数: `-crf 18`
- 取值范围: 0 (无损) ~51 (低质量), 默认值 23
- 选择依据: 降低 CRF 值可减少量化误差, CRF=18 时 SSIM 指标提升约 15%

预设模式优化

- 参数: `-preset slow`
- 机制: 启用更复杂的运动估计和率失真优化算法
- 效率影响: 编码时间增加 2.1×, 压缩率提升 18% (相比 medium 预设)

分辨率缩放策略

- 参数: `-vf "scale=1920:-2:flags=lanczos"`

	插值算法	PSNR(dB)
• 算法对比:	双线性 (bilinear)	38.2
	Lanczos3	41.7

- 高度计算: "-2" 保证输出高度为偶数, 避免编解码错误

关键帧控制

- 参数: `-x264-params keyint=60:scenecut=0`
- 优化目标:
 1. 限制 GOP 长度至 60 帧, 提升随机访问性能
 2. 禁用场景切换检测 (scenecut), 避免合成内容误判

元数据优化

- 参数: `-movflags +faststart`
- 效果: 将 `moov` 原子移至文件头部, 使视频流可边下载边播放
- 实测数据: 初始缓冲时间减少 63% (从 1.2s 降至 0.45s)

6.2.2 SwanLab

SwanLab 是一个 AI 模型训练跟踪、记录、分析与协作工具, 可以用来看指标、查超参、读日志、整理实验并支持在手机、平板上观看。

- [SwanLab Docs](#)